

Cloud Based Media File Storage Service – Detailed Project Specification (Python + React)

1) Executive Overview

A cloud-based file storage and sharing web application similar to the core features of Google Drive. The platform allows users to upload, organize, search, and share files securely using role-based access.

Goal: Build a production-grade MVP using **Python (FastAPI/Flask)** as backend and **React (or framework)** as frontend.

Primary Users: Individuals, small teams

Non-goals (MVP): Real-time co-editing, desktop sync client, office document editors

2) Feature Scope

2.1 Core (MVP)

- User authentication (Email + OAuth)
- Folder management (CRUD, hierarchy)
- File upload/download (drag & drop)
- File sharing (Viewer / Editor roles)
- Public shareable links (expiry & password optional)
- Search & filters
- Starred files
- Trash & restore (soft delete)

2.2 Phase 2+

- Version history
- File previews (image, PDF)
- Activity logs

- Tags & labels
 - Storage quota management
-

3) User Roles & Permissions

- **Owner:** Full control
- **Editor:** Upload, edit, delete
- **Viewer:** Read-only
- **Public User:** Access via shared link

All permission checks are enforced server-side.

4) User Stories (Sample)

- As a user, I can upload files and see upload progress
 - As a user, I can organize files in folders
 - As a user, I can share files with other users
 - As a user, I can restore deleted files from Trash
-

5) System Architecture

5.1 Recommended Stack

Frontend

- **React.js / Next.js / Vite**
- **Tailwind CSS**
- React Query (TanStack)
- Axios
- React Dropzone

Backend

- Python **FastAPI** (recommended) or Flask
- SQLAlchemy / SQLModel / **Supabase**
- Pydantic (validation)
- JWT Authentication

Database

- PostgreSQL (Supabase)

Storage

- Supabase Storage or AWS S3

Auth

- JWT (Access + Refresh)
 - OAuth (Google)
-

5.2 High-Level Architecture

[React Client]

|

v

[FastAPI Backend]

|

v

[PostgreSQL (Supabase)] ---- [Object Storage (S3 / Supabase)]

6) Data Model (PostgreSQL)

Core Tables

- users
- folders
- files
- file_versions
- shares
- link_shares
- stars

- activities
-

7) API Design (FastAPI)

Auth

- POST /auth/register
- POST /auth/login
- GET /auth/me

Files

- POST /files/init-upload
- POST /files/complete-upload
- GET /files/{id}
- DELETE /files/{id}

Folders

- POST /folders
- GET /folders/{id}

Sharing

- POST /shares
 - POST /public-link
-

8) Security Checklist

- JWT stored in HttpOnly cookies
 - Role-based access checks
 - Signed URLs for file access
 - Input validation with Pydantic
 - Rate limiting (SlowAPI)
-

9) Frontend UX

- Sidebar (My Drive, Shared, Trash)
 - File grid/list view
 - Upload dropzone
 - Share modal
 - Breadcrumb navigation
-

10) Deployment

Frontend

- Vercel / Netlify

Backend

- Render / Fly.io / Railway

Database & Storage

- Supabase
-

11) Project Structure

Backend (FastAPI)

backend/

├─ app/

| └─ main.py

| └─ models/

| └─ schemas/

| └─ routes/

| └─ services/

| └─ core/

```
|   └─ utils/
|   └─ requirements.txt
|   └─ .env
```

Frontend (React)

frontend/

```
|   └─ src/
|   |   └─ components/
|   |   └─ pages/
|   |   └─ services/
|   |   └─ hooks/
|   |   └─ styles/
|   └─ tailwind.config.js
|   └─ vite.config.js
```

12) 2-Week Development Schedule (Detailed Execution Plan)

This plan assumes **6–8 hours/day** and focuses on building a production-ready MVP.

Week 1 – Backend (Python: FastAPI / Flask)

Day 1 – Requirement Analysis & Database Design

- Deeply understand features (auth, files, folders, sharing)
- Finalize MVP vs Phase-2 features
- Design ER diagram (users, files, folders, shares, versions)

- Decide storage provider (Supabase Storage / AWS S3)
- Initialize Git repo and backend project

Deliverables:

- ER diagram
 - Backend repo initialized
-

Day 2 – Backend Setup & Authentication

- Setup FastAPI / Flask project structure
- Configure PostgreSQL (Supabase)
- Setup SQLAlchemy / SQLModel
- Implement:
 - User registration
 - Login
 - Password hashing (bcrypt)
 - JWT access & refresh tokens
- Middleware for protected routes

Deliverables:

- Working auth APIs
 - JWT-based auth system
-

Day 3 – File Upload & Object Storage

- Configure Supabase Storage / AWS S3 bucket
- Implement file upload flow:
 - Init upload API
 - Generate signed URL
 - Upload directly to storage
- Save file metadata in DB
- Handle file size & type validation

Deliverables:

- Secure file upload system
 - File metadata stored
-

Day 4 – Folder System & File Management APIs

- Folder CRUD APIs (nested folders)
- File operations:
 - Rename
 - Move
 - Delete (soft delete)
- Breadcrumb logic using parent_id

Deliverables:

- Fully working folder & file APIs
-

Day 5 – Sharing & Permissions

- Implement role-based access control:
 - Owner
 - Editor
 - Viewer
- Share file/folder with another user
- Public shareable links with expiry
- Permission validation middleware

Deliverables:

- Secure sharing system
-

Day 6 – Search, Trash & Optimization

- Implement search API:
 - Name-based
 - Type-based
- Implement Trash:
 - Soft delete
 - Restore logic
- Add DB indexes for performance

Deliverables:

- Search & trash working
-

Day 7 – Testing & Backend Deployment

- Test APIs using Postman
- Add basic unit tests (pytest)
- Environment variable setup
- Deploy backend (Render / Fly.io)

Deliverables:

- Live backend API
-



Week 2 – Frontend (React / Next.js)

Day 8 – Frontend Setup & Auth UI

- Initialize React (Vite / Next.js)
- Setup Tailwind CSS
- Configure Axios & React Query
- Create Login & Signup pages
- Store auth tokens securely

Deliverables:

- Auth UI integrated with backend
-

Day 9 – Dashboard & File Listing UI

- Create Google Drive–like layout
- Sidebar (My Drive, Shared, Trash)
- File & folder listing
- Breadcrumb navigation

Deliverables:

- Functional dashboard
-

Day 10 – File Upload & Preview UI

- Drag & drop upload (React Dropzone)
- Upload progress indicator
- File preview (image / pdf)

Deliverables:

- Smooth upload experience
-

Day 11 – Sharing UI & Permissions

- Share modal (email / public link)
- Permission selector (viewer/editor)
- Show shared users

Deliverables:

- Complete sharing UI
-

Day 12 – Search, Sorting & Optimization

- Search bar
- Sorting by name/date/size
- Pagination / lazy loading

Deliverables:

- Fast, optimized UI
-

Day 13 – Trash, Versioning & Final Testing

- Trash UI
- Restore & permanent delete
- (Optional) Version history UI
- End-to-end testing

Deliverables:

- Feature-complete frontend
-

Day 14 – Deployment & Polish

- Deploy frontend to Vercel
- Mobile responsiveness
- Error handling & loaders

- Final README & screenshots

Deliverables:

- Live full-stack application
-
-

13) Resources (Python + React – Best YouTube + Docs)

Best YouTube Resources (Highly Recommended)

Full-Stack Projects

- Google Drive Clone (Next.js + Supabase): <https://youtu.be/lie0cr3wESQ>
- Full Stack SaaS App (React + FastAPI): <https://youtu.be/0sOvCWFmrtA>

FastAPI (Backend)

- FastAPI Crash Course – <https://youtu.be/7t2alSnE2-I>
- FastAPI + JWT Auth – https://youtu.be/0A_GCXBCNUQ
- SQLAlchemy with FastAPI – <https://youtu.be/AhCSfuG9Jxw>

Flask (Alternative Backend)

- Flask Full Course – https://youtu.be/Z1RJmh_OqeA

React Frontend

- React Full Course – <https://youtu.be/bMknfKXIFA8>
- React Query – <https://youtu.be/novnyCaa7To>
- Tailwind CSS – <https://youtu.be/UBOj6rqRUME>

File Uploads & Storage

- AWS S3 Signed URLs – <https://youtu.be/eQAlojcArRY>
 - Supabase Storage – <https://youtu.be/WiwfiVdfRlc>
-

Official Documentation

Frontend

- React Docs – <https://react.dev>

- Next.js – <https://nextjs.org/docs>
- Tailwind CSS – <https://tailwindcss.com/docs>
- TanStack Query – <https://tanstack.com/query>
- React Dropzone – <https://react-dropzone.js.org/>

Backend (Python)

- FastAPI – <https://fastapi.tiangolo.com/>
- Flask – <https://flask.palletsprojects.com/>
- SQLAlchemy – <https://docs.sqlalchemy.org/>
- Pydantic – <https://docs.pydantic.dev/>

Database & Storage

- Supabase Docs – <https://supabase.com/docs>
- PostgreSQL – <https://www.postgresql.org/docs/>
- AWS S3 – <https://docs.aws.amazon.com/s3/>

Auth & Security

- JWT – <https://jwt.io/introduction>
 - OAuth Google – <https://developers.google.com/identity>
 - OWASP Cheat Sheets – <https://cheatsheetseries.owasp.org/>
-

14) Skills You Will Gain

- Designing scalable backend APIs
 - Secure file uploads & access control
 - Full-stack React + Python architecture
 - Cloud deployment & storage management
-

15) Final Outcome

A complete, resume-ready **Cloud Storage SaaS MVP** built using modern **Python backend** and **React frontend**, suitable for startups, internships, and product-based interviews.



This document is fully aligned with the provided PDF but adapted for Python backend architecture.